

AWS Certified Generative AI Developer — Professional

The Use Case

- Global law firm, offices in 30 countries, multiple languages
- **Corpus:** 15 million documents — case law, statutes, regulations, internal legal opinions
- **Problem:** keyword-only search; attorneys spending 15 hours/week on research with inconsistent results
- **Outcome:** research time down to 4 hours/week, sub-second queries across 15M documents, new precedents searchable within hours, ~\$20M annual saving

How The Four Tasks Differ

Task	The question it answers
1.1	How do I build the system?
1.2	How do I select and manage the model?
1.3	What happens to the data before it reaches the model?
1.4	How do I build and maintain the vector store itself?

Task 1.1 treated the vector store as one line — "you need one". Task 1.4 lives entirely inside it, and uses **four different stores at once** rather than one.

1. Vector Database Architecture

Bedrock Knowledge Bases — public legal documents

- **Fully managed:** chunking, embedding and storage all handled. You supply an S3 bucket.
- **Choose when** nothing custom is needed and you want it working quickly
- **Hierarchical organisation** by jurisdiction and legal domain (US → Federal → 9th Circuit; Contract → Breach → Remedies)
- **Structure-preserving chunking matters here** — a legal clause cut in half changes meaning

OpenSearch Service + Neural plugin — internal opinions

Two reasons this is the strongest fit for legal work:

1. **Hybrid search** — semantic and keyword combined:

Semantic:	"breaking a contract"	-> finds "breach of contract"
Keyword:	"Section 230"	-> finds exactly that

Legal work needs both. An attorney searching **"Section 230"** does not want something *similar* — they want that exact citation. Semantic search is weak on citations; keyword search is weak on concepts. Hybrid blends both scores.

- **2. Security boundaries** — fine-grained access control at index, document and field level. Internal opinions are confidential.
- **Neural plugin** integrates embedding generation and k-NN search inside OpenSearch, so you don't embed separately before sending
- Configuration involves k-NN settings and index mappings for hybrid search

RDS + S3 — structured metadata

This is not a vector store. It runs alongside one.

S3	-> the documents themselves
RDS	-> metadata (case number, court, date, judge, citations)

Some queries are **SQL problems, not semantic ones**:

"All 9th Circuit cases after 2020 where Judge Smith presided, that haven't been overturned"

Embedding that question is pointless — it is joins and filters. Typical pattern: filter in RDS to narrow candidates, then run vector search over what remains.

DynamoDB — live regulatory updates

- Single-digit millisecond latency, very high throughput
- **Choose when** the query is simple ("latest version of this regulation") and speed dominates

- **DynamoDB does not do vector search by itself** — it is paired with a vector library such as Faiss or Pinecone, storing metadata and embeddings while the library handles similarity
- Requires deliberate partition/sort key design for high-dimensional vectors

Store comparison

Store	Strength	Use when
Knowledge Bases	Fully managed, least effort	Standard RAG, no customisation needed
OpenSearch	Hybrid search, fine-grained security, tunable	Complex queries needing keyword + semantic
RDS + S3	SQL joins, complex filters, relationships	Structured metadata queries
DynamoDB	Sub-ms latency, huge throughput	Simple real-time lookups

Multi-model architecture — query routing

"text of Section 230"	-> DynamoDB (exact, fast)
"9th Circuit, post-2020, ..."	-> RDS then vector
"contract breach concepts"	-> Knowledge Bases (semantic)
"our internal opinion on ..."	-> OpenSearch (secure + hybrid)

Results are then **aggregated and reranked by precedential value** — a Supreme Court case outranks a district court case even at slightly lower similarity. **In legal work, authority matters, not just similarity.**

2. Metadata Frameworks

Vector search only sees **meaning**. It cannot tell you a case is from the wrong jurisdiction, out of date, or **already overturned** — an overturned case and a valid one have almost identical embeddings, because the meaning is the same. Only metadata makes that distinction.

What to capture

Standard: creation date, modification history, authorship, content type, version.

Legal-specific (from the transcript): document type, jurisdiction, practice area, and **precedential value** — court hierarchy plus whether the case is still good law.

S3 object metadata and its limits

- S3 offers **object metadata** (key-value) and **object tags** (filtering, lifecycle)
- **Limits:** metadata capped around 2 KB; only 10 tags per object
- Transcript guidance: use S3's capability but **address its limitations with supplementary storage** — some metadata in S3, the rest in RDS or DynamoDB

Hierarchical tagging — taxonomies

Practice Area
Litigation
Commercial
Contract Disputes
Breach of Warranty

- **Tag inheritance:** searching "Litigation" returns everything beneath it, without tagging each level separately
- Lets attorneys query at whichever level of specificity fits their question
- Transcript defines three taxonomies: **practice area, procedural context, jurisdiction**

Automated extraction — Comprehend + custom ML

- 15 million documents cannot be tagged by hand
- **Comprehend:** entities, key phrases, sentiment, custom entity recognition
- **Custom ML is also needed** because Comprehend doesn't understand law — **citation extraction** ("410 U.S. 113"), legal concept identification, document classification
- Runs as a pipeline over every incoming document

Cross-reference metadata — relationships

Case A cites Case B
Case C overturns Case A
Amendment Y modifies Statute X

You find the perfect case — but someone overturned it in 2015. **Vector search cannot tell you this.** Only a relationship graph can. This is precisely what RDS (or a graph database) is for: `citations`, `overturned_by` — joins a vector store cannot perform.

Metadata lives in two places

	Contains	Purpose
Vector store	Chunk-level, limited	Filtering inside the search
RDS	Document-level, rich	SQL joins, relationships

Ingestion order: metadata is extracted first, then written to both — document-level metadata to RDS, chunk-level metadata alongside each vector. As in Task 1.1, **metadata added later requires full re-ingestion.**

3. High-Performance Architecture

15 million documents, sub-second responses. A single flat index would search the entire corpus on every query.

Sharding

- A **shard** is a slice of an index, distributed across nodes — enabling parallel search and reducing per-node load
- **Transcript's two-level approach:** shard by practice area, then sub-shard by time period
- A contract query touches only contract shards; a "recent" query touches only the current time shard. **Everything else is never read.**
- **Shard count** is decided from vector dimensionality, dataset size and query patterns — too few makes each shard heavy, too many adds coordination overhead
- **Replicas** are copies, for redundancy and read throughput — a separate decision from shard count

Multi-index

- Sharding splits one index; multi-index creates **separate indices** per domain (`legal-contract-2024` , `legal-ip-2024`)
- **The advantage sharding can't give:** each index can be tuned independently — the transcript specifies *custom similarity algorithms tuned for specific language patterns and citation structures*, since tax and IP language differ
- Old indices can be moved to cold storage or dropped

Hierarchical indexing — coarse to fine

Level 1: 15M docs -> clusters -> pick nearest 3 clusters
Level 2: search fully within those clusters
Result: 500K vectors searched instead of 15M

This is the principle behind approximate nearest neighbour search (e.g. **HNSW**). **Trade-off:** slight recall loss in exchange for large speed gains.

Vector compression — quantization

A 1024-dimension float32 vector is ~4 KB. 15M chunks is roughly **60 GB of vectors alone**, and RAM is expensive.

Technique	Effect
Scalar quantization	float32 → int8 per dimension; roughly 4× memory reduction
Product quantization	Split the vector into sub-vectors replaced by codebook entries; 16–32× reduction

Trade-off: less memory and faster queries, at some cost to accuracy. How much must be measured — legal search can tolerate very little.

Caching — ElastiCache

- Cache **query embeddings, search results**, and frequently accessed vectors
- Transcript: *pre-computes vector similarities for common query patterns* — answers ready before the query arrives
- Decisions: cache size, eviction policy (LRU), refresh/TTL
- **Invalidation matters in legal:** a new precedent makes a cached result wrong

Monitoring

- **CloudWatch** — query latency, cache hit rate, cluster health
- **X-Ray** — where the time actually went across the pipeline (as in Task 1.1: CloudWatch says *how slow*, X-Ray says *where*)

Query path with everything applied

Query
-> cache hit? -> return immediately
-> route: which index (practice area + time)
-> that index's shards
-> hierarchical: coarse then fine
-> similarity over compressed vectors
-> results

4. Integration Components

Ingestion — two different patterns

Pattern	When
Lambda + EventBridge	Single document, event-driven, near real-time. One connector per DMS (iManage, NetDocuments) since each API differs.
AWS Glue ETL	Bulk transformation of structured sources (knowledge management systems, wikis). Extract → transform to vector-compatible format, preserving relationships and metadata → load.

Glue rather than Lambda for bulk work because it involves thousands of records and exceeds Lambda's 15-minute limit.

Unified search interface

```
Attorney -> API Gateway -> Lambda router
      |
      KB / OpenSearch / RDS / DynamoDB
      |
      aggregate + rerank
      |
      Foundation Model
```

Three jobs: **query routing**, **aggregation**, and **reranking**.

Why reranking is required: scores from different stores aren't comparable — 0.85 from OpenSearch doesn't mean 0.85 from DynamoDB. And legal relevance isn't similarity alone; the transcript reranks on **precedential value**.

Security — the highest-stakes part

Concept	Why it matters
Client-matter boundaries	Client A's data must never surface for Client B's attorney. A legal obligation, not just policy.
Privilege designations	Attorney-client privileged material shown to the wrong person can destroy the privilege — the document becomes unusable in court.

- **Cognito** — identity and which matters the attorney works on, carried as token claims
- **IAM** — resource-level access

Document-level filtering must happen inside the search, not after it. Search first and filter later, and the model has already seen material it shouldn't have. Same pattern as Marie's jurisdiction filter in Task 1.1 — and as there, the permission comes from the **signed token**, never from user input.

Tracing and error handling

- **X-Ray** — with four stores and several Lambdas, tracing is the only way to find which hop is slow
- **SQS** — queue documents for ingestion; provides natural rate limiting
- **Step Functions** — retries, error catching, dead-letter queue. One bad document goes to the DLQ instead of halting the batch.

5. Data Maintenance

A vector store is a snapshot; the world keeps moving. New cases publish daily, regulations change, precedents get overturned. Transcript target: **new precedents searchable within hours**.

Incremental updates — EventBridge + Lambda

- Only the changed document is re-embedded — reprocessing 15M documents every time is not viable
- **Three operations:** new → add; changed → **delete old chunks, then insert new**; deleted → remove
- The middle one is commonly forgotten, leaving stale duplicate chunks behind

Real-time change detection

Trigger source	Fires on
S3 event notifications	File uploaded or deleted
DynamoDB Streams	Row changed
Database triggers	RDS change

All are push-based, not polling. Used in the transcript for legislative changes, where yesterday's regulation may be wrong today.

Scheduled full refresh — EventBridge + AWS Batch

Why incremental updates aren't enough:

- **The embedding model changed.** Old and new embeddings live in different spaces and **cannot be compared** — the whole corpus must be re-embedded
- **Fragmentation** from repeated add/delete degrades performance
- **Drift** — small inconsistencies accumulate
- **Chunking strategy changed** — everything must be re-chunked

AWS Batch rather than Lambda because this runs for hours over the full corpus.

Step Functions — orchestrating the sync

```
extract content
-> preprocess
-> extract metadata (Comprehend)
-> chunk
-> embed
-> write to vector store
-> update RDS metadata
-> invalidate cache
```

Cache invalidation is the commonly missed step — a new precedent lands but attorneys keep getting the cached older answer.

Health monitoring — CloudWatch

Metric	Tells you
Sync latency	Time from publication to searchable — this is how the "within hours" target is measured
Retrieval relevance	Are results still useful?
Embedding quality	Missing or malformed embeddings
Index health	Fragmentation, size

Automated remediation — Lambda + SNS

```
missing embeddings  -> Lambda re-embeds
outdated content    -> trigger refresh
fragmented index    -> trigger reindex
                    -> SNS alerts the team
```

The four maintenance patterns

Pattern	When	Services
Incremental	Routine daily changes	EventBridge + Lambda
Real-time	Time-sensitive content	S3 events, DynamoDB Streams, DB triggers
Scheduled rebuild	New embedding model, drift, fragmentation	EventBridge + AWS Batch
Orchestrated sync	Multi-step update process	Step Functions

Service limits and feature availability change. Verify specific numbers against current AWS documentation before the exam.